

LeetCode 1456: Max Number of Vowels in a Substring

Bloomberg Wayfair

Rule of Thumb: Counting elements within a fixed substring generally implies using the **Sliding Window** technique.

Approach 1: Sliding Window

 $O(n)$ Time | $O(1)$ Space

Build the first window of size k as a reference to calculate the initial vowel count. Then, slide the window across the string by adding the element entering the window and removing the element leaving the window. Update the maximum count accordingly.

```
class Solution {
public:
    int maxVowels(string s, int k) {
        int n=s.length();
        int count=0;
        for(int i=0;i<k;i++){
            if(s[i]=='a' || s[i]=='e' || s[i]=='i' || s[i]=='o' || s[i]=='u'){
                count++;
            }
        } //build the first window for a reference
        int maxu=count;

        for(int j=k;j<n;j++){ //see how other windows are doing
            if(s[j]=='a' || s[j]=='e' || s[j]=='i' || s[j]=='o' || s[j]=='u'){count++;}
            if(s[j-k]=='a' || s[j-k]=='e' || s[j-k]=='i' || s[j-k]=='o' || s[j-k]=='u'){count--;}
            maxu=max(maxu,count);
        }

        return maxu;
    }
};
```

Approach 2: Prefix Count Array

$O(n)$ Time | $O(n)$ Space

Make a prefix count array which keeps track of how many vowels have been seen up to that index. This allows for constant time querying of the number of vowels in any substring window.

Index (Char)	a	b	c	i	i	i	d
Prefix Count	1	1	1	2	3	4	4

$\text{prefix}[r + 1] - \text{prefix}[l] = \text{Number of vowels in window } [l, r]$

```
int n = s.size();
vector<int> prefix(n+1,0);

// Build prefix vowel count
for(int i=0;i<n;i++)
{
    prefix[i+1] = prefix[i] + isVowel(s[i]);
}

int ans = 0;

// Check every window
for(int l=0;l+k-1<n;l++)
{
    int r = l + k - 1;
    int vowels = prefix[r+1] - prefix[l];
    ans = max(ans, vowels);

    // Early exit
    if(ans == k)
        return k;
}

return ans;
```